UNIVERSIDAD NACIONAL DE COLOMBIA

Programación Orientada a Objetos 2026-2S

ACTIVIDAD 2

Estudiante: Thomas Ramírez Jaramillo

Docente: Walter Hugo Arboleda Mazo

Asignatura: Programación Orientada a Objetos

Periodo: 2026-2S

Universidad Nacional de Colombia - Sede Medellín

Contenido de la actividad

La actividad contiene los cinco ejercicios solicitados, cada uno con su código fuente en Java y su diagrama de clases.

URL del repositorio en GitHub:
https://github.com/Thomasote/Actividad2_POO

El archivo README.md incluido en la carpeta del proyecto contiene los datos solicitados para el repositorio.

URL del ejercicio 2.1:
https://github.com/Thomasote/Actividad2_POO/blob/main/codigo/Persona.java

URL del ejercicio 2.2:
https://github.com/Thomasote/Actividad2_POO/blob/main/codigo/Planeta.java

URL del ejercicio 2.3:
https://github.com/Thomasote/Actividad2_POO/blob/main/codigo/Automovil.java

URL del ejercicio 2.4:
https://github.com/Thomasote/Actividad2_POO/blob/main/codigo/PruebaFiguras.java

URL del ejercicio 2.5:
https://github.com/Thomasote/Actividad2_POO/blob/main/codigo/CuentaBancaria.java

Ejercicio 2.1 - Persona

Se modela una persona con nombre, apellido, documento y año de nacimiento. Se crean dos personas y se muestran sus datos.

Código fuente

```
public class Persona {
    private String nombre;
    private String apellido;
    private String documento;
    private int anioNacimiento;

    public Persona(String nombre, String apellido, String documento,
                   int anioNacimiento) {
        this.nombre=nombre;
        this.apellido = apellido;
        this.documento=documento;
        this.anioNacimiento = anioNacimiento;
    }

    public void imprimir() {
        System.out.println("Nombre: " + nombre);
        System.out.println("Apellido: " + apellido);
        System.out.println("Documento: " + documento);
        System.out.println("Anio de nacimiento: " + anioNacimiento);
    }

    public static void main(String[] args) {
        Persona persona1 = new Persona("Thomas", "Ramirez", "000001", 2007);
        Persona persona2=new Persona("Daniel", "Lopez", "000002", 2005);
        persona1.imprimir();
        System.out.println();
        persona2.imprimir();
    }
}
```

Diagrama de clases

Persona
- nombre : String - apellido : String - documento : String - anioNacimiento : int
+ Persona(String nombre, String apellido, String documento, int anioNacimiento) + imprimir() : void + main(String[] args) : void

Ejercicio 2.2 - Planeta

Se modelan dos planetas con sus atributos, se calcula su densidad y se determina si son exteriores al cinturón de asteroides.

Código fuente

```
public class Planeta {
    public enum TipoPlaneta { GASEOSO, TERRESTRE, ENANO }

    private String nombre = null;
    private int satelites=0;
    private double masa = 0;
    private double volumen=0;
    private int diametro = 0;
    private int distanciaSol=0;
    private TipoPlaneta tipo;
    private boolean visible = false;

    public Planeta(String nombre, int satelites, double masa,
                   double volumen, int diametro, int distanciaSol,
                   TipoPlaneta tipo, boolean visible) {
        this.nombre=nombre;
        this.satelites = satelites;
        this.masa=masa;
        this.volumen = volumen;
        this.diametro=diametro;
        this.distanciaSol = distanciaSol;
        this.tipo=tipo;
        this.visible = visible;
    }

    public double calcularDensidad() {
        return masa/volumen;
    }

    public boolean esExterior() {
        double distanciaUA = distanciaSol*1000000.0/149597870;
        return distanciaUA>3.4;
    }

    public void imprimir() {
        System.out.println("Planeta: " + nombre);
        System.out.println("Satelites: " + satelites);
        System.out.println("Masa (kg): " + masa);
        System.out.println("Volumen (km3): " + volumen);
        System.out.println("Diametro (km): " + diametro);
        System.out.println("Distancia al Sol (millones de km): " + distanciaSol);
        System.out.println("Tipo: " + tipo);
        System.out.println("Visible a simple vista: " + visible);
    }

    public static void main(String[] args) {
        Planeta tierra = new Planeta("Tierra", 1, 5.9736E24,
                1.08321E12, 12742, 150, TipoPlaneta.TERRESTRE, true);
        Planeta jupiter=new Planeta("Jupiter", 79, 1.899E27,
                1.4313E15, 139820, 778, TipoPlaneta.GASEOSO, true);

        tierra.imprimir();
        System.out.println("Densidad (kg/km3): " + tierra.calcularDensidad());
        System.out.println("Es exterior: " + tierra.esExterior());
        System.out.println();
        jupiter.imprimir();
        System.out.println("Densidad (kg/km3): " + jupiter.calcularDensidad());
        System.out.println("Es exterior: " + jupiter.esExterior());
    }
}
```

Diagrama de clases

Planeta
- nombre : String - satelites : int - masa : double - volumen : double - diametro : int - distanciaSol : int - tipo : TipoPlaneta - visible : boolean
+ Planeta(String nombre, int satelites, double masa, double volumen, int diametro, int distanciaSol, TipoPlaneta tipo, boolean visible) + calcularDdensidad() : double + esExterior() : boolean + imprimir() : void + main(String[] args) : void

<<enumeration>> TipoPlaneta
GASEOSO, TERRESTRE, ENANO

Planeta utiliza la enumeración TipoPlaneta.

Ejercicio 2.3 - Automóvil

Se modela un automóvil con sus atributos y métodos de consulta y modificación. Se prueba la aceleración, la desaceleración, el frenado y el tiempo estimado de llegada.

Código fuente

```
public class Automovil {
    public enum TipoCombustible {
        GASOLINA, BIOETANOL, DIESEL, BIODIESEL, GAS_NATURAL
    }
    public enum TipoAutomovil {
        CIUDAD, SUBCOMPACTO, COMPACTO, FAMILIAR, EJECUTIVO, SUV
    }
    public enum Color {
        BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE, AZUL, VIOLETA
    }

    private String marca;
    private int modelo;
    private double motor;
    private TipoCombustible combustible;
    private TipoAutomovil tipo;
    private int puertas;
    private int asientos;
    private int velocidadMaxima;
    private Color color;
    private int velocidadActual;

    public Automovil(String marca, int modelo, double motor,
                     TipoCombustible combustible, TipoAutomovil tipo,
                     int puertas, int asientos, int velocidadMaxima,
                     Color color, int velocidadActual) {
        if (velocidadMaxima<0 || velocidadActual<0
                || velocidadActual>velocidadMaxima) {
            throw new IllegalArgumentException("Velocidades invalidas");
        }
        this.marca=marca;
        this.modelo = modelo;
        this.motor=motor;
        this.combustible = combustible;
        this.tipo=tipo;
        this.puertas = puertas;
        this.asientos=asientos;
        this.velocidadMaxima = velocidadMaxima;
        this.color=color;
        this.velocidadActual = velocidadActual;
    }

    public String getMarca() {
        return marca;
    }

    public void setMarca(String marca) {
        this.marca=marca;
    }

    public int getModelo() {
        return modelo;
    }

    public void setModelo(int modelo) {
        this.modelo=modelo;
    }

    public double getMotor() {
        return motor;
    }

    public void setMotor(double motor) {
        this.motor=motor;
    }
```

```
public TipoCombustible getCombustible() {
    return combustible;
}

public void setCombustible(TipoCombustible combustible) {
    this.combustible=combustible;
}

public TipoAutomovil getTipo() {
    return tipo;
}

public void setTipo(TipoAutomovil tipo) {
    this.tipo=tipo;
}

public int getPuertas() {
    return puertas;
}

public void setPuertas(int puertas) {
    this.puertas=puertas;
}

public int getAsientos() {
    return asientos;
}

public void setAsientos(int asientos) {
    this.asientos=asientos;
}

public int getVelocidadMaxima() {
    return velocidadMaxima;
}

public void setVelocidadMaxima(int velocidadMaxima) {
    if (velocidadMaxima<0 || velocidadMaxima<velocidadActual) {
        System.out.println("Velocidad no permitida");
        return;
    }
    this.velocidadMaxima=velocidadMaxima;
}

public Color getColor() {
    return color;
}

public void setColor(Color color) {
    this.color=color;
}

public int getVelocidadActual() {
    return velocidadActual;
}

public void setVelocidadActual(int velocidadActual) {
    if (velocidadActual<0 || velocidadActual>velocidadMaxima) {
        System.out.println("Velocidad no permitida");
        return;
    }
    this.velocidadActual=velocidadActual;
}

public void acelerar(int aumento) {
    if (aumento<0) {
        System.out.println("El aumento no puede ser negativo");
    } else if (aumento>velocidadMaxima-velocidadActual) {
        System.out.println("No se puede superar la velocidad maxima");
    } else {
        velocidadActual = velocidadActual+aumento;
    }
}
```

```
    public void desacelerar(int disminucion) {
        if (disminucion<0) {
            System.out.println("La disminucion no puede ser negativa");
        } else if (disminucion>velocidadActual) {
            System.out.println("La velocidad no puede quedar negativa");
        } else {
            velocidadActual=velocidadActual-disminucion;
        }
    }

    public void frenar() {
        velocidadActual=0;
    }

    public double calcularTiempoLlegada(double distancia) {
        if (!Double.isFinite(distancia) || distancia<0) {
            throw new IllegalArgumentException("Distancia invalida");
        }
        if (distancia==0) {
            return 0;
        }
        if (velocidadActual==0) {
            throw new IllegalStateException("El automovil esta detenido");
        }
        return distancia/velocidadActual;
    }

    public void imprimir() {
        System.out.println("marca: " + marca);
        System.out.println("modelo: " + modelo);
        System.out.println("motor: " + motor);
        System.out.println("combustible: " + combustible);
        System.out.println("tipo: " + tipo);
        System.out.println("puertas: " + puertas);
        System.out.println("asientos: " + asientos);
        System.out.println("velocidadMaxima: " + velocidadMaxima);
        System.out.println("color: " + color);
        System.out.println("velocidadActual: " + velocidadActual);
    }

    public static void main(String[] args) {
        Automovil carro = new Automovil("Renault", 2020, 1.6,
                TipoCombustible.GASOLINA, TipoAutomovil.COMPACTO,
                4, 5, 180, Color.AZUL, 0);
        carro.imprimir();
        carro.setVelocidadActual(100);
        System.out.println("Velocidad: " + carro.getVelocidadActual());
        carro.acelerar(20);
        System.out.println("Velocidad: " + carro.getVelocidadActual());
        carro.desacelerar(50);
        System.out.println("Velocidad: " + carro.getVelocidadActual());
        System.out.println("Tiempo para 140 km (horas): "
                + carro.calcularTiempoLlegada(140));
        carro.frenar();
        System.out.println("Velocidad: " + carro.getVelocidadActual());
    }
}
```

Diagrama de clases

Automovil
- marca : String - modelo : int - motor : double - combustible : TipoCombustible - tipo : TipoAutomovil - puertas : int - asientos : int - velocidadMaxima : int - color : Color - velocidadActual : int
+ Automovil(String marca, int modelo, double motor, TipoCombustible combustible, TipoAutomovil tipo, int puertas, int asientos, int velocidadMaxima, Color color, int velocidadActual) + getMarca() : String + setMarca(String marca) : void + getModelo() : int + setModelo(int modelo) : void + getMotor() : double + setMotor(double motor) : void + getCombustible() : TipoCombustible + setCombustible(TipoCombustible combustible) : void + getTipo() : TipoAutomovil + setTipo(TipoAutomovil tipo) : void + getPuertas() : int + setPuertas(int puertas) : void + getAsientos() : int + setAsientos(int asientos) : void + getVelocidadMaxima() : int + setVelocidadMaxima(int velocidadMaxima) : void + getColor() : Color + setColor(Color color) : void + getVelocidadActual() : int + setVelocidadActual(int velocidadActual) : void + acelerar(int aumento) : void + desacelerar(int disminucion) : void + frenar() : void + calcularTiempoLlegada(double distancia) : double + imprimir() : void + main(String[] args) : void

<<enumeration>> TipoCombustible
GASOLINA, BIOETANOL, DIESEL, BIODIESEL, GAS_NATURAL

<<enumeration>> TipoAutomovil
CIUDAD, SUBCOMPACTO, COMPACTO, FAMILIAR, EJECUTIVO, SUV

<<enumeration>> Color
BLANCO, NEGRO, ROJO, NARANJA, AMARILLO, VERDE, AZUL, VIOLETA

Automovil utiliza las enumeraciones TipoCombustible, TipoAutomovil y Color.

Ejercicio 2.4 - Figuras geométricas

Se crean un círculo, un rectángulo, un cuadrado y un triángulo rectángulo. Se calculan sus áreas y perímetros, además de la hipotenusa y el tipo del triángulo.

Código fuente

Circulo.java

```
public class Circulo {
    private int radio;

    public Circulo(int radio) {
        if (radio<=0) {
            throw new IllegalArgumentException("Medidas positivas requeridas");
        }
        this.radio=radio;
    }

    public double calcularArea() {
        return Math.PI*radio*radio;
    }

    public double calcularPerimetro() {
        return 2*Math.PI*radio;
    }
}
```

Cuadrado.java

```
public class Cuadrado {
    private int lado;

    public Cuadrado(int lado) {
        if (lado<=0) {
            throw new IllegalArgumentException("Medidas positivas requeridas");
        }
        this.lado=lado;
    }

    public double calcularArea() {
        return (double) lado*lado;
    }

    public double calcularPerimetro() {
        return 4.0*lado;
    }
}
```

Rectangulo.java

```
public class Rectangulo {
    private int base;
    private int altura;

    public Rectangulo(int base, int altura) {
        if (base<=0 || altura<=0) {
            throw new IllegalArgumentException("Medidas positivas requeridas");
        }
        this.base=base;
        this.altura=altura;
    }

    public double calcularArea() {
        return (double) base*altura;
    }
```

```
    public double calcularPerimetro() {
        return 2.0*(base+altura);
    }
}
```

TrianguloRectangulo.java

```
public class TrianguloRectangulo {
    private int base;
    private int altura;

    public TrianguloRectangulo(int base, int altura) {
        if (base<=0 || altura<=0) {
            throw new IllegalArgumentException("Medidas positivas requeridas");
        }
        this.base=base;
        this.altura=altura;
    }

    public double calcularArea() {
        return (double) base*altura/2;
    }

    public double calcularPerimetro() {
        return base+altura+calcularHipotenusa();
    }

    public double calcularHipotenusa() {
        return Math.hypot(base, altura);
    }

    public String determinarTipo() {
        if (base==altura) {
            return "Isosceles";
        }
        return "Escaleno";
    }
}
```

PruebaFiguras.java

```
public class PruebaFiguras {
    public static void main(String[] args) {
        Circulo circulo=new Circulo(3);
        Rectangulo rectangulo = new Rectangulo(4, 6);
        Cuadrado cuadrado=new Cuadrado(5);
        TrianguloRectangulo triangulo = new TrianguloRectangulo(3, 4);

        System.out.println("Circulo");
        System.out.println("Area: " + circulo.calcularArea());
        System.out.println("Perimetro: " + circulo.calcularPerimetro());
        System.out.println("Rectangulo");
        System.out.println("Area: " + rectangulo.calcularArea());
        System.out.println("Perimetro: " + rectangulo.calcularPerimetro());
        System.out.println("Cuadrado");
        System.out.println("Area: " + cuadrado.calcularArea());
        System.out.println("Perimetro: " + cuadrado.calcularPerimetro());
        System.out.println("Triangulo rectangulo");
        System.out.println("Area: " + triangulo.calcularArea());
        System.out.println("Perimetro: " + triangulo.calcularPerimetro());
        System.out.println("Hipotenusa: " + triangulo.calcularHipotenusa());
        System.out.println("Tipo: " + triangulo.determinarTipo());
    }
}
```

Diagrama de clases

Circulo
- radio : int
+ Circulo(int radio) + calcularArea() : double + calcularPerimetro() : double

Cuadrado
- lado : int
+ Cuadrado(int lado) + calcularArea() : double + calcularPerimetro() : double

Rectangulo
- base : int - altura : int
+ Rectangulo(int base, int altura) + calcularArea() : double + calcularPerimetro() : double

TrianguloRectangulo
- base : int - altura : int
+ TrianguloRectangulo(int base, int altura) + calcularArea() : double + calcularPerimetro() : double + calcularHipotenusa() : double + determinarTipo() : String

PruebaFiguras
+ main(String[] args) : void

PruebaFiguras utiliza las clases Circulo, Rectangulo, Cuadrado y TrianguloRectangulo. Un triángulo rectángulo puede ser isósceles o escaleno, pero no equilátero.

Ejercicio 2.5 - Cuenta bancaria

Se modela una cuenta bancaria que permite consultar el saldo, consignar y retirar dinero, verificando que no se retire más del saldo disponible.

Código fuente

```
public class CuentaBancaria {
    public enum TipoCuenta { AHORROS, CORRIENTE }

    private String nombres;
    private String apellidos;
    private String numeroCuenta;
    private TipoCuenta tipo;
    private double saldo=0;

    public CuentaBancaria(String nombres, String apellidos,
                          String numeroCuenta, TipoCuenta tipo) {
        this.nombres=nombres;
        this.apellidos = apellidos;
        this.numeroCuenta=numeroCuenta;
        this.tipo = tipo;
    }

    public void imprimir() {
        System.out.println("Titular: " + nombres + " " + apellidos);
        System.out.println("Numero de cuenta: " + numeroCuenta);
        System.out.println("Tipo: " + tipo);
        System.out.println("Saldo: " + saldo);
    }

    public double consultarSaldo() {
        return saldo;
    }

    public boolean consignar(double valor) {
        if (!Double.isFinite(valor) || valor<=0
                || !Double.isFinite(saldo+valor)) {
            System.out.println("Valor de consignacion invalido");
            return false;
        }
        saldo=saldo+valor;
        return true;
    }

    public boolean retirar(double valor) {
        if (!Double.isFinite(valor) || valor<=0) {
            System.out.println("El retiro debe ser un valor positivo");
            return false;
        }
        if (valor>saldo) {
            System.out.println("Saldo insuficiente");
            return false;
        }
        saldo = saldo-valor;
        return true;
    }

    public static void main(String[] args) {
        CuentaBancaria cuenta=new CuentaBancaria("Thomas", "Ramirez",
                "00123456", TipoCuenta.AHORROS);
        cuenta.imprimir();
        cuenta.consignar(200000);
        cuenta.consignar(300000);
        System.out.println("Saldo: " + cuenta.consultarSaldo());
        cuenta.retirar(400000);
        System.out.println("Saldo: " + cuenta.consultarSaldo());
        cuenta.retirar(150000);
        System.out.println("Saldo final: " + cuenta.consultarSaldo());
    }
}
```

Diagrama de clases

CuentaBancaria
- nombres : String - apellidos : String - numeroCuenta : String - tipo : TipoCuenta - saldo : double
+ CuentaBancaria(String nombres, String apellidos, String numeroCuenta, TipoCuenta tipo) + imprimir() : void + consultarSaldo() : double + consignar(double valor) : boolean + retirar(double valor) : boolean + main(String[] args) : void

<<enumeration>> TipoCuenta
AHORROS, CORRIENTE

CuentaBancaria utiliza la enumeración TipoCuenta.